Algorithm Complexity

Determining an algorithm's complexity means

- see *how fast* (time complexity) an algorithm is, based on the size of the input, and
- check *how much memory* (space complexity) an algorithm requires, based on the size of the input.

Usually time is a biggest concern

- Many algorithms require a fixed amount of space.

Input Size

Time and space complexity is a function of the input size.

How we characterize input size depends:

- Sorting: number of input items
- Multiplication: total number of bits
- Graph algorithms: number of nodes and edges

How to measure time complexity?

Analyze algorithm analytically

- How?
  - Count the number of basic operations of an algorithm

    Read, write, compare, assign, jump, arithmetic operations (increment, decrement, add, subtract, multiply, divide), open, close, logical operations (not/complement, AND, OR, XOR), …

  - Summarize the count

Example#1

```
1.    x=0
2.    for (i=0; i < n; i++)
3.        for (j=0, j < m; j++)
4.            x = x + 1
```

The body of the loop of line 2 is executed *n* times
The body of the loop of line 3 is executed *m* times
Count= *n*m*

Example#2

```
1.  for (i=0; i<n; i++)
2.      for (j=0, j<i; j++)
3.          x = x + 1
```

$$\text{Count} = \sum_{i=0}^{n-1}\sum_{j=0}^{i-1} 1 = \sum_{i=0}^{n-1} i = (n-1)n/2$$

Counting Primitive operations

By inspecting the pseudocode, we can determine the maximum number of primitive operations executed by an algorithm, as a function of the input size

Algorithm ***arrayMax***(***A***, ***n***)	# operations
currentMax ← ***A***[0]	2
for ***i*** ← 1 **to** ***n*** – 1 **do**	$c_1 n$
if ***A***[*i*] > ***currentMax*** **then**	$c_2(n-1)$
currentMax ← ***A***[***i***]	$c_3(n-1)$
{ increment counter ***i*** }	$c_4(n-1)$
return ***currentMax***	1
Total:	$(c_1+c_2+c_3+c_4)n - (c_2+c_3+c_4-3)$

Estimating Running Time

Algorithm ***arrayMax*** executes $cn - d$ primitive operations in the worst case, where c and d are constants.

Define:
a = Time taken by the fastest primitive operation
b = Time taken by the slowest primitive operation

Let $T(n)$ be worst-case time of ***arrayMax***. Then

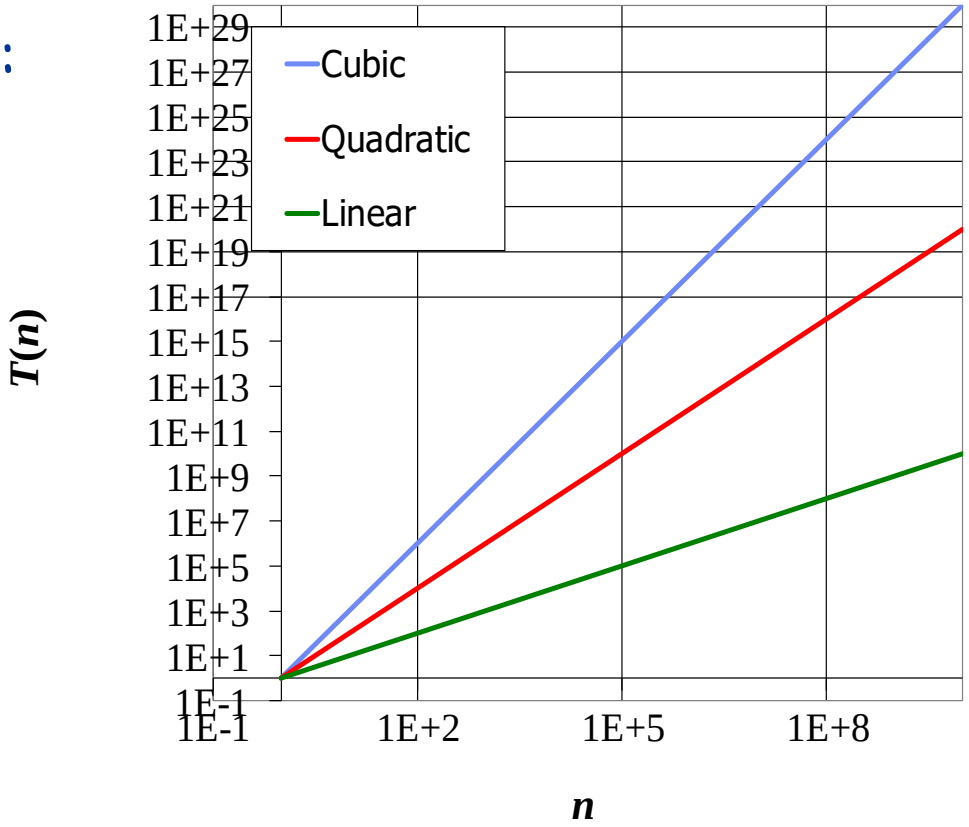
$$a\,(cn - d) \leq T(n) \leq b(cn - d)$$

Hence, the running time $T(n)$ is bounded by two linear functions.

Growth Rates

Growth rates of functions:

- Linear ≈ n
- Quadratic ≈ n^2
- Cubic ≈ n^3



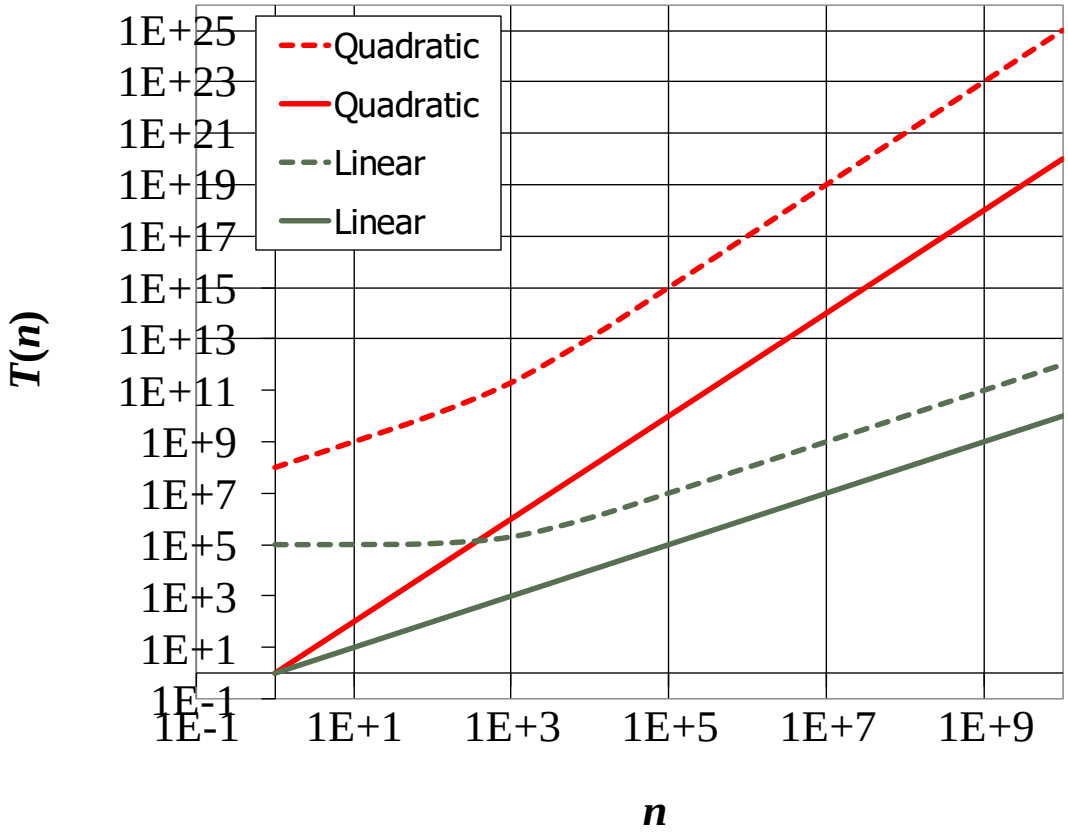
Constant Factors

The growth rate is not affected by

- constant factors or
- lower-order terms

Examples

- $10^2n + 10^5$ is a linear function
- $10^5n^2 + 10^8n$ is a quadratic function



Asymptotic Order

Big-O Notation

Given functions $f(n)$ and $g(n)$, we say that $f(n)$ is $O(g(n))$ if there are positive constants c and n_0 such that $f(n) \leq cg(n)$ for $n \geq n_0$

Example: $2n + 10$ is $O(n)$

$2n + 10 \leq cn$

$\Rightarrow (c - 2)\, n \geq 10$

$\Rightarrow n \geq 10/(c - 2)$

Pick $c = 3$ and $n_0 = 10$

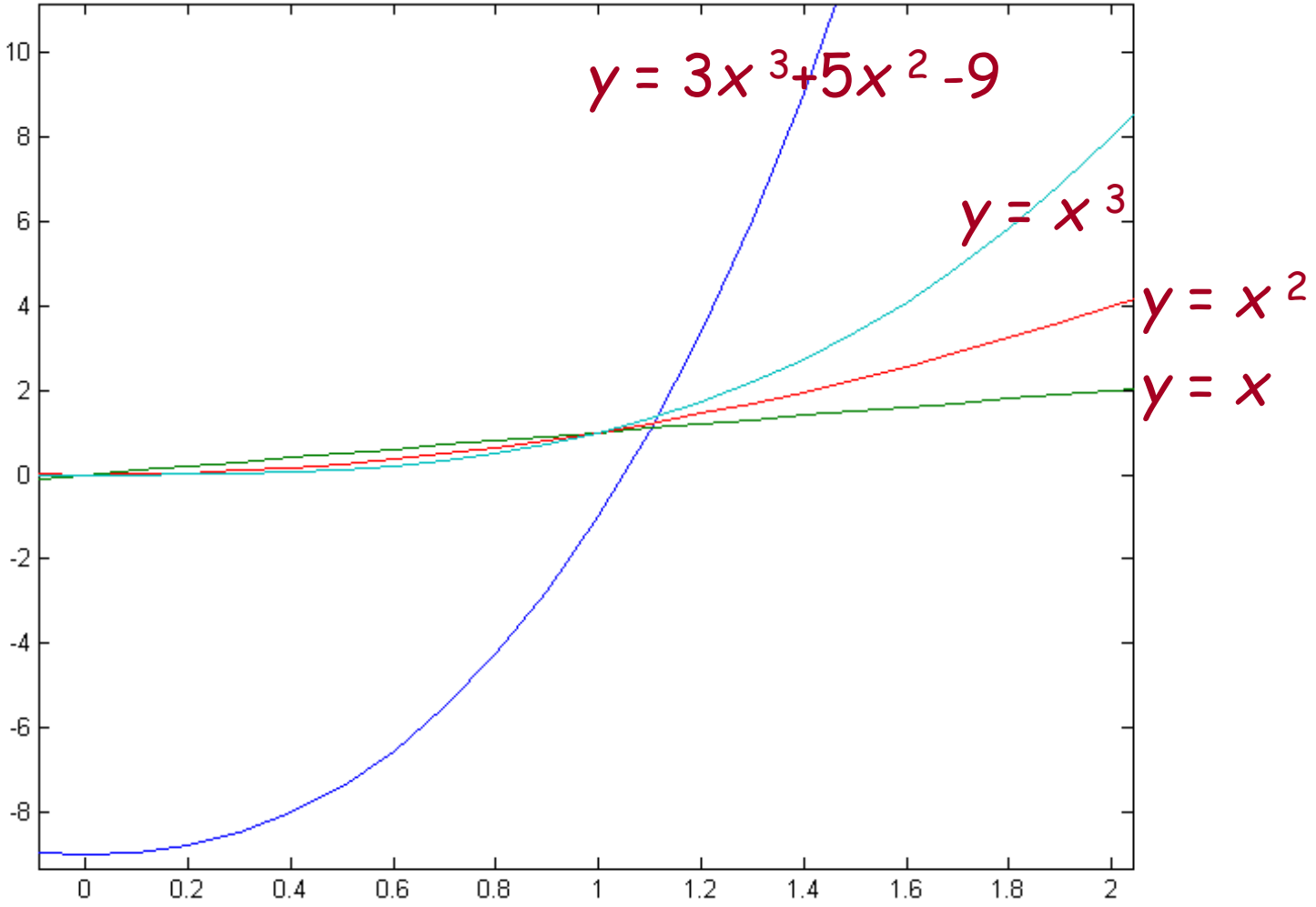
Intuitive Notion of Big-O

Asymptotic notation captures behavior of functions for large values of x.

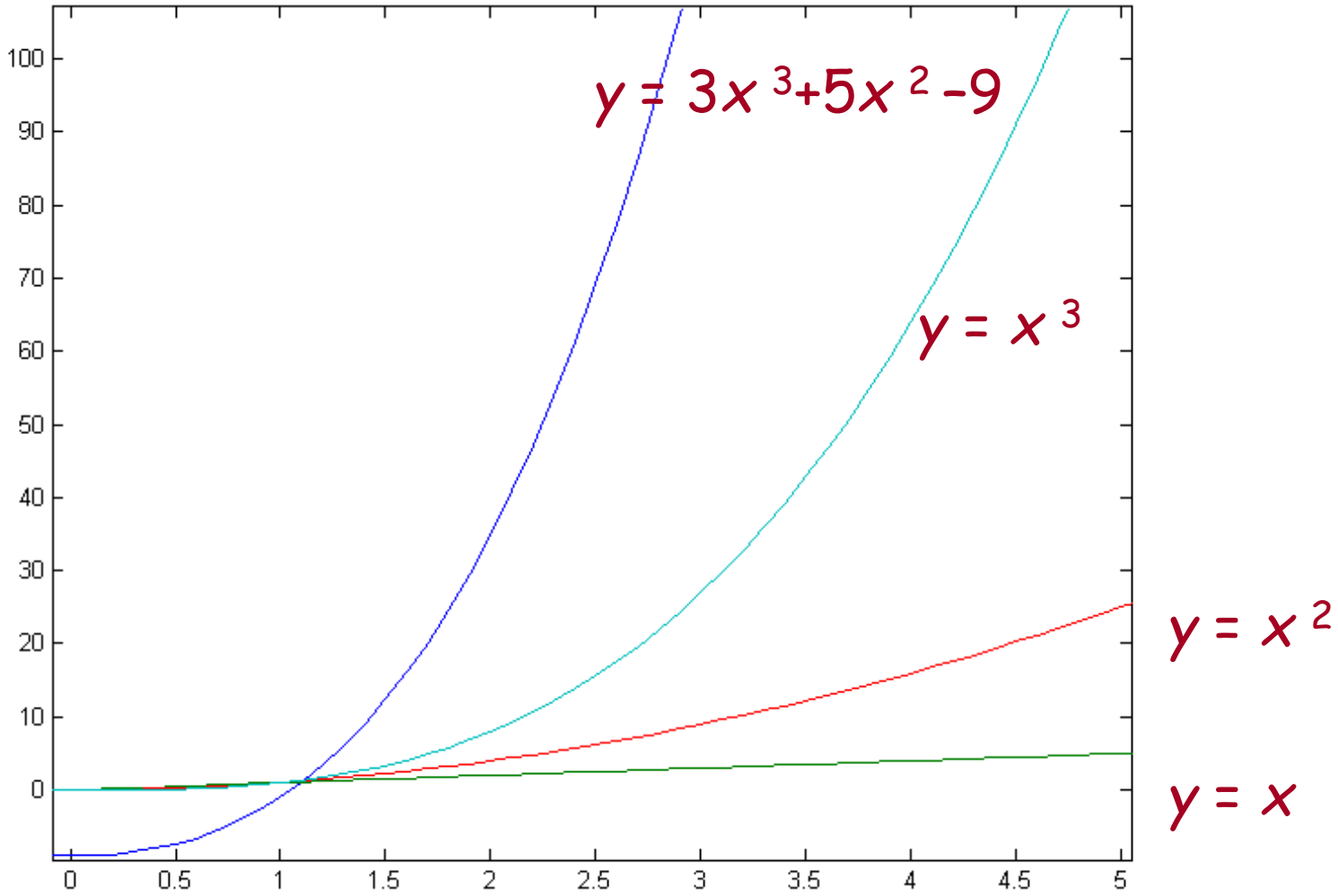
Dominating term of $3x^3+5x^2-9$ is x^3.

As x becomes larger and larger, other terms become insignificant and only x^3 remains in the picture.

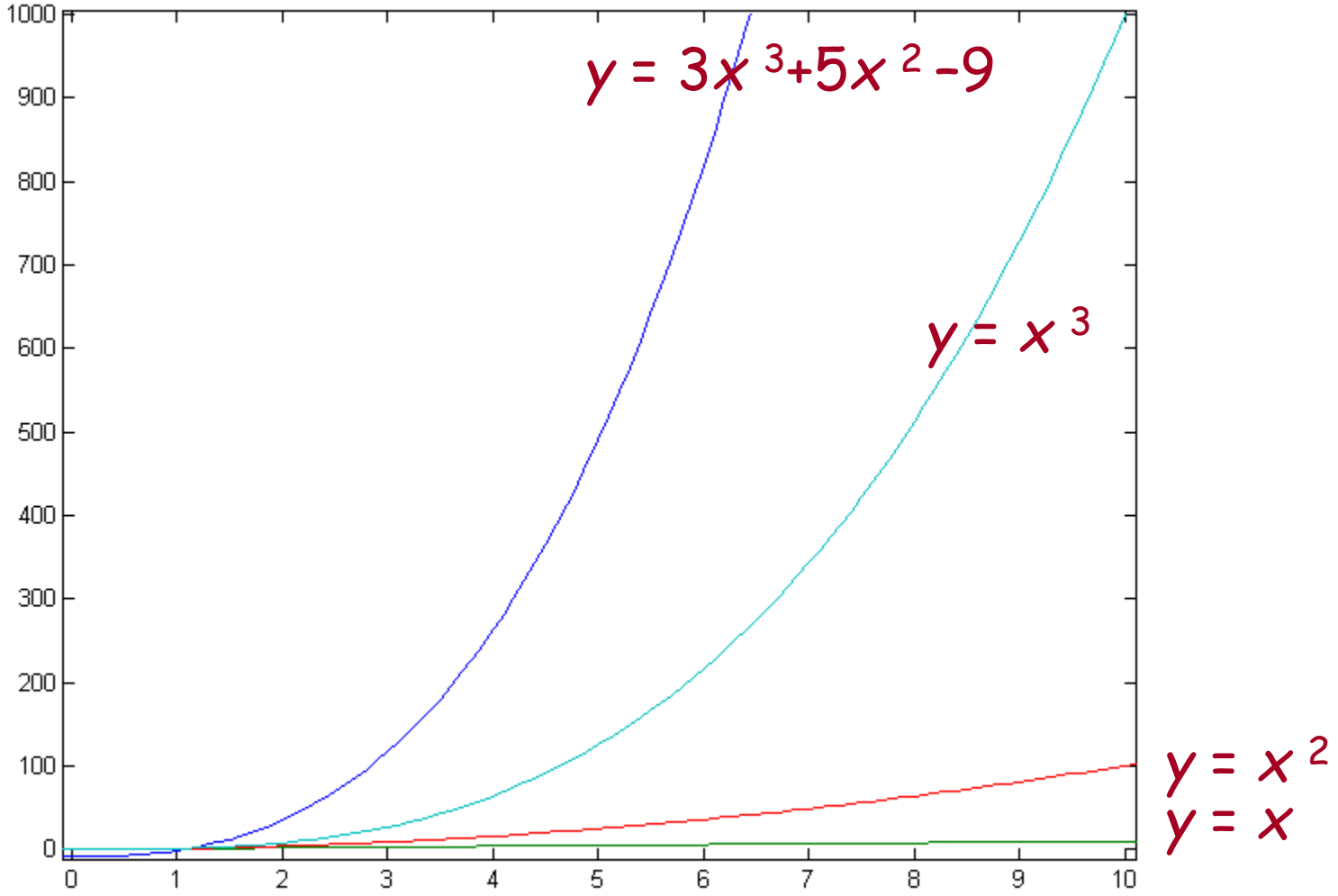
Intuitive Notion of Big-O *domain* [0,2]



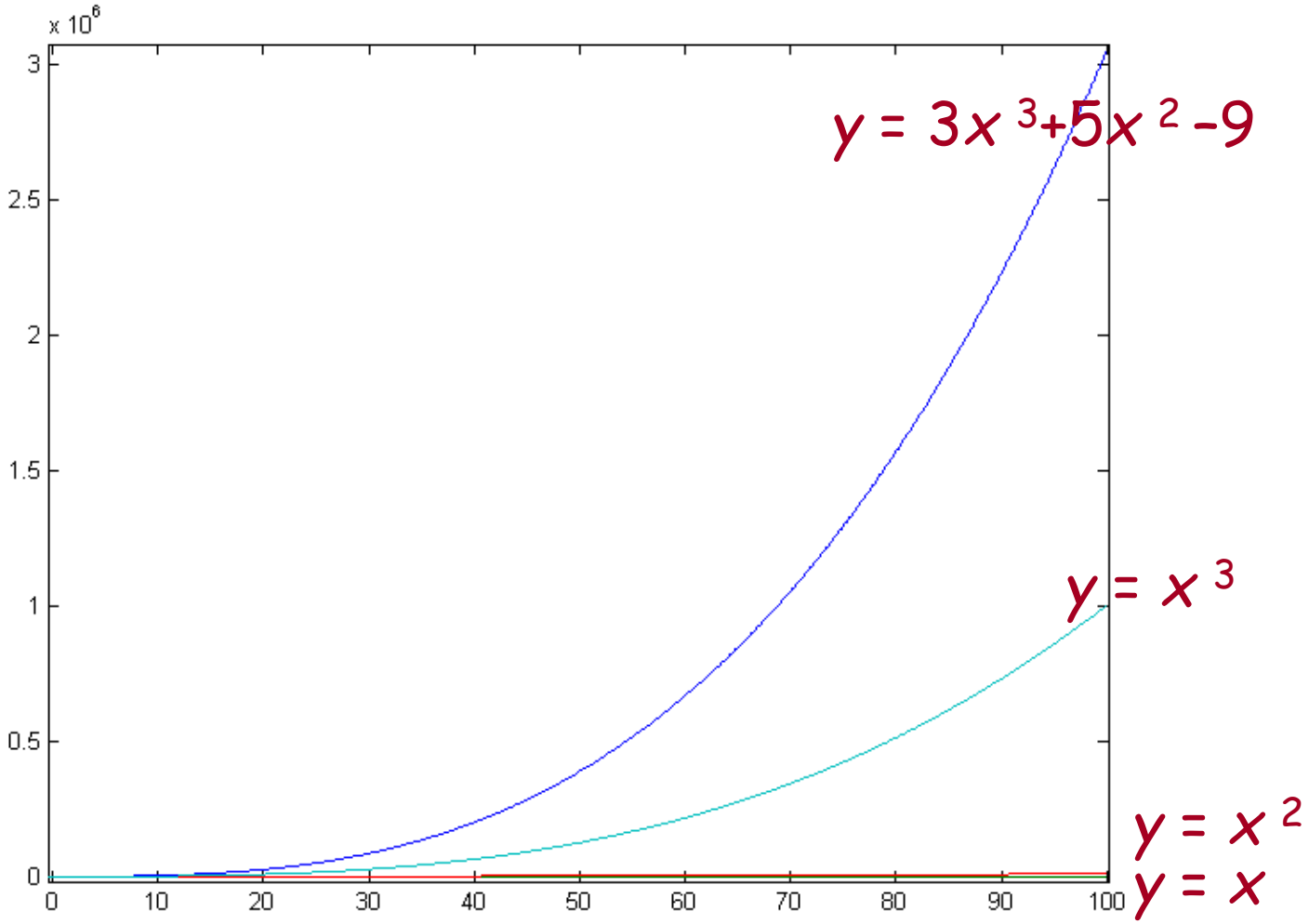
Intuitive Notion of Big-O *domain* [0,5]



Intuitive Notion of Big-O *domain* [0,10]



Intuitive Notion of Big-O *domain* [0,100]



Linear Time: O(n)

Linear time. Running time is at most a constant factor times the size of the input.

Example: Compute maximum of n numbers $a_1, \ldots, a_n$.

```
max = a_1
for i = 2 to n {
   if (a_i > max)
      max = a_i
}
```

Task

Merge. Combine two sorted lists $A = a_1, a_2, \ldots, a_n$ with $B = b_1, b_2, \ldots, b_n$ into sorted whole.

```
i = 1, j = 1
while (both lists are non-empty) {
   if (a_i ≤ b_j) append a_i to output list and increment i
   else append b_j to output list and increment j
}
append remainder of nonempty list to output list
```

Sub-linear Time

Binary search. Given a sorted array A, check if a given number p belongs to the array.

O(log n) solution. Probe particular entries in the array.

```
Bsearch(A, p, l, h):
   if (h < l) return false;
   mid = (l + h)/2;
   if (A[mid] > p) return Bsearch(A, p, l, mid - 1);
   else if (A[mid] < p) return Bsearch(A, p, mid + 1, h);
   else return true;
```

Big-O and Growth Rate

The big-O notation gives an upper bound on the growth rate of a function

The statement "$f(n)$ is $O(g(n))$" means that the growth rate of $f(n)$ is no more than the growth rate of $g(n)$

We can use the big-O notation to rank functions according to their growth rate

	$f(n)$ is $O(g(n))$	$g(n)$ is $O(f(n))$
$g(n)$ grows more	Yes	No
$f(n)$ grows more	No	Yes
Same growth	Yes	Yes

Relatives of Big-O

big-Omega

f(n) is $\Omega(g(n))$ if there is a constant c > 0 and an integer constant $n_0 \geq 1$ such that $f(n) \geq c \cdot g(n)$ for $n \geq n_0$

big-Theta

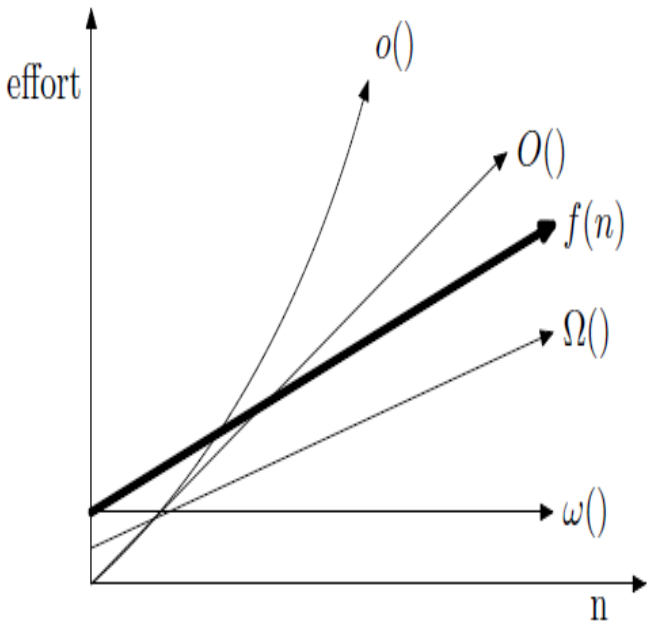
f(n) is $\Theta(g(n))$ if there are constants c' > 0 and c'' > 0 and an integer constant $n_0 \geq 1$ such that $c' \cdot g(n) \leq f(n) \leq c'' \cdot g(n)$ for $n \geq n_0$

little-o

f(n) is o(g(n)) if, for any constant c > 0, there is an integer constant $n_0 \geq 1$ such that $f(n) < c \cdot g(n)$ for $n \geq n_0$

little-omega

f(n) is $\omega(g(n))$ if, for any constant c > 0, there is an integer constant $n_0 \geq 1$ such that $f(n) > c \cdot g(n)$ for $n \geq n_0$



Big-Omega

$f(x) = \Omega(g(x)) \leftrightarrow g(x) = O(f(x))$
so $f(x)$ dominates $g(x)$.

Example: $f(x)= 8x^3 + 5x^2 + 7$, $g(x)=x^3$

Big-Theta - exact order

Big-Θ: domination in both directions.

$$f(x) = \Theta(g(x)) \leftrightarrow f(x) = O(g(x)) \wedge f(x) = \Omega(g(x))$$

Example: $3x^2 + 8x\log x$ is $\Theta(x^2)$

$0 \leq 8x\log x \leq 8x^2$, implies that $3x^2 + 8\log x \leq 11x^2$ for $x>1$.
That is, $3x^2 + 8\log x$ is $O(x^2)$

Also, x^2 is $O(3x^2 + 8x\log x)$. That is, $3x^2 + 8\log x$ is $\Theta(x^2)$

Properties

Transitivity.

- If $f = O(g)$ and $g = O(h)$ then $f = O(h)$.
- If $f = \Omega(g)$ and $g = \Omega(h)$ then $f = \Omega(h)$.
- If $f = \Theta(g)$ and $g = \Theta(h)$ then $f = \Theta(h)$.

Additivity.

- If $f = O(h)$ and $g = O(h)$ then $f + g = O(h)$.
- If $f = \Omega(h)$ and $g = \Omega(h)$ then $f + g = \Omega(h)$.
- If $f = \Theta(h)$ and $g = \Theta(h)$ then $f + g = \Theta(h)$.

At a glance!

$\Theta(1)$	Constant complexity
$\Theta(\log n)$	Logarithmic complexity
$\Theta(n)$	Linear complexity
$\Theta(n^c)$	Polynomial (for any c) complexity
$\Theta(c^n)$	Exponential (for $c>1$) complexity

Worst Case, Best Case, and Average Case

The worst-case complexity of the algorithm is the function defined by the **maximum** number of operations taken on any instance of size n. In other words, the maximum running time over all possible inputs of a given size.

The best-case complexity of the algorithm is the function defined by the **minimum** number of operations taken on any instance of size n. In other words, the minimum running time over all the possible inputs of a given input size

The average-case complexity of the algorithm is the function defined by the number of operations taken on random instance of size n.